

紙芝居アプリ内部仕様書

Copyright © 2026 Hiroya Kubo. この文書は [CC BY-SA 4.0](#) で提供します。

この文書は、TMPose紙芝居の汎用アプリ SB3、成果物、SB3・台本変換ビルダー、検証、公開の内部仕様を、現在の実装に対応させて記録します。アプリを変更する手順は [ソフトウェア開発者向け資料](#)、台本の外部仕様は [台本DSLマニュアル](#)と [コマンドリファレンス](#)を参照してください。

対象アプリ/DSL: `kamishibai=3.1`

過去のバージョンからの変更は [history.md](#) を参照してください。

1. 文書の範囲と実装基準

アプリ内部構造の正本は `app/project.source.json` です。本書の `target`、変数、`list`、`broadcast`、`hat`、カスタムブロック定義は、このファイルから抽出した現在の構造を 記載しています。配布用 `kamishibai.sb3` は `app/` から生成する成果物であり、本書の 調査元にはしません。

1.1 実装スナップショット

項目	件数
target (Stageを含む)	8
block	1444
event hat	39
カスタムブロック定義	40
Scratch変数	6
Scratch list	11
broadcast message	18
静的な runtime variable 名	16
静的な thread variable 名	36
TurboWarp 機能拡張	12

block ID は展開ソース内の対応箇所を特定するために掲載します。TurboWarp でブロックを 作り直すと ID は変わり得るため、外部仕様や永続 ID として使用しません。

1.2 使用する機能拡張

ID	役割	取得形態
<code>sipconsole</code>	デバッグ用 console	Gallery
<code>lmsTempVars2</code>	runtime variable / thread variable	Gallery
<code>strings</code>	文字列処理	Gallery
<code>kubohiroyaassetmanager</code>	画像・音声の登録と Loading 進捗	埋め込み
<code>tmpose</code>	カメラ姿勢認識	埋め込み
<code>localStorage</code>	台本のローカル保存	Gallery
<code>kubohiroyatextlines</code>	行単位の台本処理	埋め込み
<code>kubohiroyaruntimeexpression</code>	分岐条件式の評価	埋め込み
<code>kubohiroyaasyncinput</code>	key / touch 入力と scene 移動	埋め込み
<code>lmsTimers</code>	<code>wait</code> と時間ベース actor action のタイマー	Gallery
<code>files</code>	外部台本ファイルの選択	Gallery
<code>text</code>	テキスト描画・アニメーション	Gallery

埋め込み拡張の由来、固定 commit、SHA-256 は `app/embedded-extensions.json` を正本とし、更新方法は [sb3-toolchain](#) のワークフロー に従います。

2. 成果物プロファイル

紙芝居の成果物は、台本と物語固有アセットをどこに保持するかで分けます。

プロファイル	例	台本	物語固有アセット	主な用途
<code>generic</code>	<code>kamishibai.sb3</code>	非埋め込み	非埋め込み	本体が配布する汎用雛形
<code>editor</code>	<code>_urashima.sb3</code>	非埋め込み	埋め込み	物語作成者の編集・動作確認
<code>player</code>	<code>urashima.sb3</code>	埋め込み	埋め込み	配布・再生、Packager Web 版

`generic` は `app/` から生成し、特定の物語を含めません。builder API と CLI が受け付ける `profile` は `editor` または `player` です。

`editor` と `player` は同じベース SB3、台本、アセットロックから生成します。両者の 変換済み台本とアセット参照を分岐させません。`player` は組み込み台本を予約変数へ保存し、タイトル操作後にファイル選択なしで開始します。

`player` へ台本とアセットを組み込んでも、TMPose モデル、カメラ、外部サービスまで自動的にオフライン化されるわけではありません。残るオンライン依存は成果物 manifest と 公開ページへ明記します。

3. SB3・台本変換ビルダー

3.1 導入

利用可能なバージョンを確認し、消費側で明示的に固定します。

```
npm view @kubohiroya/tmpose-kamishibai version
pnpm add --save-exact @kubohiroya/tmpose-kamishibai@<VERSION>
```

生成した lockfile を commit し、CI では `pnpm install --frozen-lockfile` を使います。

3.2 CLI

```
pnpm exec tmpose-kamishibai build-sb3 \
  --base kamishibai.sb3 \
  --script source.txt \
  --assets assets.lock.json \
  --output dist/sample \
  --profile editor
```

`--output` は拡張子を含まないベース名です。次の 3 ファイルを同じ transaction として 生成します。

```
dist/sample.sb3
dist/sample.txt
dist/sample.manifest.json
```

オプション	意味
<code>--allow-file-root DIR</code>	<code>file:</code> の許可ルートを追加。複数回指定可能
<code>--allow-http</code>	平文HTTPを明示的に許可
<code>--timeout-ms N</code>	1 リクエストのタイムアウト
<code>--max-asset-bytes N</code>	1 アセットの最大バイト数
<code>--max-script-bytes N</code>	組み込み台本の最大バイト数
<code>--max-redirects N</code>	HTTPリダイレクト上限

完全な一覧は `pnpm exec tmpose-kamishibai --help` で確認します。

3.3 JavaScript API

```
import {
  Sb3BuilderError,
  buildSb3Bundle,
  validateAssetManifest,
  validateBundle,
} from '@kubohiroya/tmpose-kamishibai/builder';

const result = await buildSb3Bundle({
  baseSb3: 'kamishibai.sb3',
  sourceScript: 'source.txt',
  assetManifest: 'assets.lock.json',
  outputDirectory: 'dist',
  outputName: 'sample',
  profile: 'editor',
});

console.log(result.outputPaths);
```

`baseSb3` と `sourceScript` にはファイルパスまたは `file:` URL を指定できます。`assetManifest` にはファイルパス、`file:` URL、または検証対象の JavaScript オブジェクトを指定できます。相対 `file:` を含むオブジェクトでは `manifestBaseDirectory` も指定します。

ネットワーク・ファイル取得は `allowedFileRoots`、`allowHttp`、`requestTimeoutMs`、`maxAssetBytes`、`maxRedirects` で制限できます。`player` の組み込み台本上限は `maxEmbeddedScriptBytes` で変更できます。

`buildSb3Bundle` は `manifest` と `outputPaths` を返します。入力・アセット・出力の問題は `Sb3BuilderError` として処理段階とアセット情報を保持します。

3.4 アセットマニフェスト

入力 manifest は `formatVersion: 1` と 1 件以上の `assets` を持ちます。

```
{
  "formatVersion": 1,
  "assets": [
    {
      "name": "forest",
      "uri": "file:assets/forest.svg",
      "kind": "backdrop",
      "target": "@stage",
      "sb3Name": "森",
      "contentType": "image/svg+xml",
      "dataFormat": "svg",
      "size": 1234,
      "sha256": "<64文字の16進数>",
      "license": "CC-BY-4.0: https://creativecommons.org/licenses/by/4.0/",
      "metadata": {
        "bitmapResolution": 1,
        "rotationCenterX": 240,
        "rotationCenterY": 180
      }
    }
  ]
}
```

kind	target	変換後の台本参照
backdrop	@stage	backdrop:<sb3Name>
costume	スプライト名	costume:<target>:<sb3Name>
stageSound	@stage	sound:@stage:<sb3Name>
spriteSound	スプライト名	sound:<target>:<sb3Name>

DSL 名、同一 target 内の SB3 名、既存 SB3 のアセット名は重複できません。`license` には素材の ライセン
スまたは利用条件の識別情報と参照先を記録します。

3.5 安全性と再現性

- `file:` は既定でmanifestのディレクトリ以下だけを許可し、`..` やsymlinkによる脱出を拒否する
- HTTPSを既定とし、平文HTTPは明示的に許可した場合だけ取得する
- Content-Type、実サイズ、ロック済みサイズ、SHA-256、timeout、redirectを検証する
- ZIP entry 順、timestamp、圧縮設定、JSON表現を固定する
- SB3、変換済み台本、出力manifestの対応を確定前に再検証する
- 3成果物を一時領域で生成し、すべて成功した場合だけ置換する
- 失敗時は既存成果物を保持または復元する

同じ入力、固定依存、設定から生成したSB3、台本、manifestはbit-for-bitで一致しなければなりません。

4. SB3の構成

4.1 target一覧

target	種別	役割	初期costume/ sound
Stage	Stage	初期化、台本解析、scene/action実行、カメラ、入力、遷移を統括	Title, Stars
Actor	sprite 雛形	物語上の登場人物ごとにcloneされ、移動・見た目・音・時間actionを実行	button1 / pop
prompt	UI sprite	操作案内、pose案内、台本エラーをAsset Managerのcostumeで表示	ui-placeholder
openButton	UI sprite	外部台本ファイルを選択してstartStoryへ渡す	ui-placeholder
reloadButton	UI sprite	保存済みの直前の台本を再読込する	ui-placeholder
showTitleButton	UI sprite	menuからtitleへ戻す	ui-placeholder
Loading	UI sprite	Asset Managerの読込開始・進捗・完了に合わせてcostumeを表示	loading / Chirp
LoadingBubbleAnchor	UI sprite	Loading進捗メッセージ用のspeech bubble位置を固定	loading-bubble-anchor

Actorの本体は非表示で、cloneだけを登場人物として表示します。prompt、3つのmenu button、Loading、LoadingBubbleAnchorの実画像は、台本のui.*設定または組み込みfallbackからAsset Managerへ登録します。

4.2 target間の責務

Stage は台本を行へ分解し、`commandList`、`sceneList`、`actionList` へ段階的に変換します。Stage action は自分で実行し、Actor action は共有 runtime variable へ引数を置いて `execActorAction` を broadcast します。Actor clone は自分の `actorName` と `actionTarget` を照合し、対象になった clone だけが action を実行します。

UI sprite は表示状態を `showMenu` / `hideMenu`、`showPrompt` / `hidePrompt`、Asset Manager の Loading message で受け取ります。台本の実行状態を UI sprite 側へ複製せず、Stage を状態の所有者とします。

5. 変数と list

変数は、SB3 へ永続化される Scratch 変数 / list、thread ごとの一時値、project 全体で共有する runtime variable の 3 種類に分けます。

5.1 Scratch 変数

所有者	変数	初期値	役割
Stage	ポーズ認識	0	pose 認識中の表示・互換用状態
Stage	チャージ	0	pose 成立までの charge 表示・互換用状態
Stage	actionIndex	1	現在処理する <code>actionList</code> の位置
Stage	poseIndex	1	現在処理する <code>poseList</code> の位置
Stage	<code>__tmpose_embedded_script</code>	空文字	<code>player</code> profile の組み込み台本予約領域
Actor	actorName	<code>_template_</code>	clone が担当する台本上の actor 名

`__tmpose_embedded_script` は Stage に一つだけ存在し、monitor を持ちません。`generic` と `editor` では空、`player` では builder が変換済み台本を設定します。

5.2 Scratch list

すべて Stage 所有で、汎用 SB3 では空の初期状態です。

list	役割
skinList	actor action 中の costume 候補
poseList	pose action 中の認識 label 候補
soundList	actor action 中の sound 候補
actionList	現在 scene の action 列
sceneList	台本から抽出した scene block
commandList	scene の前に評価する設定 command
actorList	台本から生成する actor 定義
assetList	登録する asset 定義
durationList	pose 候補ごとの継続時間
sceneLabelList	scene label と index の対応
lines	Text Lines で分割した台本行

5.3 runtime variable

静的な名前を持つ runtime variable は次の 16 個です。

変数	生存期間／役割
<code>script</code>	読み込んだ変換済み台本。title、reload、 <code>startStory</code> 間で共有
<code>version</code>	台本の <code>kamishibai</code> version
<code>startSceneIndex</code>	台本で指定した開始 scene
<code>sceneIndex</code>	現在実行中の scene index
<code>actionTarget</code> , <code>actionCommand</code> , <code>actionParam</code> , <code>actionParam2</code>	Stage から Actor clone へ渡す action envelope
<code>nextSceneLabel</code>	key/touch 入力及要求した遷移先 scene label
<code>skipMode</code>	<code>Space</code> 、 <code>Right</code> 、 <code>Down</code> による未消費の進行要求
<code>skipContext</code>	<code>title</code> 、 <code>action</code> 、 <code>pose</code> 、 <code>scene</code> のどの境界及要求を消費できるか
<code>poseRecog</code> , <code>poseCharge</code> , <code>poseIdle</code>	pose 認識のしきい値、charge 時間、idle 時間
<code>loadingCostume</code>	Loading sprite へ適用する costume 名
<code>message</code>	Loading bubble へ表示する現在の進捗文言

このほか、`exec command %s %s` は DSL で指定された runtime variable 名を動的に設定します。分岐条件は `branch:<branchName>` という名前で保存します。この 2 系列は入力から名前が決まるため、静的な 16 個には数えません。

5.4 thread variable

カスタムブロック呼出しごとに分離され、呼出し終了後に共有状態として残さない値です。

用途	名前
共通 loop・文字列処理	index, length, line, lineIndex, name, value, key, keyValue
台本解析	sceneBlock, sceneLabel, sceneLabellist, condition, conditionList
asset/actor 生成	asset, assetList, resourceId, actor, actorName, skin
action 実行	action, actionResult, actionListResult, stageActionResult, actorActionResult
command/branch/入力	commandIndex, branchIndex, keyId, hasRead
cover	cover, coverBackground, coverBgm
Actor clone	x, y, scale
その他	durationList, sceneIndex

runtime variable と同名の `sceneIndex` thread variable は、カスタムブロック内の局所的な 引数・計算値です。project 全体の現在 scene は runtime variable 側だけを正本とします。

6. event、カスタムブロック、呼出し関係

6.1 event hat 一覧

`procedures_definition` と、接続されていない reporter block は event hat に含めません。「直接の下流」は hat から到達するカスタムブロック呼出しと broadcast です。

Stage

ID	trigger	直接の下流
iM	green flag	stop camera preview, stop pose recog, stop camera, hide all actors; showTitle 送信
i;	key space	showCover 送信
i}	key down arrow	finishTimedActorAction 送信、skipMode=Down
jb	key right arrow	finishTimedActorAction 送信、skipMode=Right
jX	startStory 受信	start camera, create sceneList, exec scene # %s with %s, create asset, create actor
j/	stopStory 受信	stop camera, stop pose recog, show cover; deleteAllActors, showMenu 送信
j?	debugTestCamera 受信	TMPoseのcamera previewを直接確認
kD	showCover 受信	show cover; hidePrompt, deleteAllActors 送信
l=	Stage click	組み込み台本の有無に応じて showCover または startStory 送信
l[	showTitle 受信	実行contextをclearし、hidePrompt, deleteAllActors 送信
m~	stopKeyInput 受信	Async Inputの全listenerを停止
nx	stopTouchInput 受信	Async Inputの全listenerを停止

Actor

ID	trigger	直接の下流
nY	execActorAction 受信	isTimeBasedAction, wait for actor action %s seconds
oH	deleteAllActors 受信	cloneを削除
oJ	clone 開始	actorName、位置、scaleをruntime envelopeから初期化
actorFinishHat	finishTimedActorAction 受信	対象actorとskipModeを照合して時間actionを完了

UI sprite

target	ID	trigger	直接の下流
prompt	oS	showPrompt 受信	案内 costume を表示
prompt	oV	hidePrompt 受信	非表示
prompt	oX	invalidScript 受信	エラー costume を表示
openButton	o!	green flag	非表示
openButton	o%	sprite click	file 選択後に hideMenu , startStory 送信
openButton	o*	hideMenu 受信	非表示
openButton	o,	showMenu 受信	ui.open skin で表示
reloadButton	o.	green flag	非表示
reloadButton	o:	hideMenu 受信	非表示
reloadButton	o=	sprite click	保存済み台本で hideMenu , startStory 送信
reloadButton	o[	showMenu 受信	台本が保存済みなら表示
showTitleButton	o`	green flag	非表示
showTitleButton	ol	hideMenu 受信	非表示
showTitleButton	o~	sprite click	hideMenu , showTitle 送信
showTitleButton	pb	showMenu 受信	title 以外なら表示
Loading	pf	green flag	非表示
Loading	pm	assetLoadingStarted 受信	Loading costume を表示
Loading	pj	assetLoadingProgress 受信	costume を循環
Loading	ph	assetLoadingCompleted 受信	非表示、完了 sound
LoadingBubbleAnchor	loadingBubbleFlag	green flag	非表示、bubble を clear
LoadingBubbleAnchor	loadingBubbleStarted	assetLoadingStarted 受信	anchor を表示

target	ID	trigger	直接の下流
LoadingBubbleAnchor	loadingBubbleProgress	assetLoadingProgress 受信	runtime variable <code>message</code> を say
LoadingBubbleAnchor	loadingBubbleCompleted	assetLoadingCompleted 受信	bubble を clear して非表示

6.2 カスタムブロック定義一覧

引数名はprototypeの `argumentnames`、warpはprototypeの `mutation.warp` から取得します。「直接の呼出し」には定義内で呼ぶ別のカスタムブロックだけを示し、broadcastは明記します。

初期化・parse・共通処理

target	ID	定義	引数	warp	直接の呼出し/broadcast
Stage	c:	init skinList with %s	commaSeparatedText	yes	selectValue # %s separated by %s from %s
Stage	c_	init poseList with %s	commaSeparatedText	yes	selectValue # %s separated by %s from %s
Stage	dc	init soundList with %s	commaSeparatedText	yes	selectValue # %s separated by %s from %s
Stage	gH	init durationList with %s	commaSeparatedText	no	selectValue # %s separated by %s from %s
Stage	eM	selectValue # %s separated by %s from %s	index, separator, text	yes	—
Stage	hI	substr of %s after %s	text, firstDelim	no	—
Stage	dL	min %s %s	valueA, valueB	yes	—
Stage	d)	exec command %s %s	key, value	no	substr of %s after %s, selectValue # %s separated by %s from %s, setTMPoseURL with %s; invalidScript
Stage	e+	create sceneList	—	yes	selectValue # %s separated by %s from %s
Stage	fe	create asset	—	no	selectValue...; assetLoadingStarted/Progress/Completed
Stage	fv	create actor	—	no	selectValue # %s separated by %s from %s

camera・pose

target	ID	定義	引数	warp	直接の呼出し／broadcast
Stage	dQ	setTMPoseURL with %s	URL	no	—
Stage	dU	start camera	—	no	—
Stage	dX	stop camera	—	no	—
Stage	eD	start pose recog	—	no	—
Stage	dG	stop pose recog	—	no	—
Stage	dY	rate of pose recog	—	no	—
Stage	d!	label of pose recog	—	no	—
Stage	d%	start camera preview	—	no	—
Stage	dC	stop camera preview	—	no	—
Stage	dk	exec pose action %s	actorName	no	start camera preview, start pose recog, exec pose %s, stop pose recog, stop camera preview; showPrompt, hidePrompt
Stage	f[	exec pose %s	actorName	no	change skin..., min..., rate of pose recog

scene・action・actor

target	ID	定義	引数	warp	直接の呼出し/broadcast
Stage	fB	exec scene # %s with %s	sceneIndex , sceneData	no	selectValue # %s separated by %s from %s , substr of %s after %s , exec command %s %s , exec actionList , hide all actors ; invalidScript
Stage	e=	exec actionList	—	no	exec action %s
Stage	fC	exec action %s	action	no	exec stage action %s , exec actor action %s
Stage	gP	exec stage action %s	action	no	touchInputToChangeScene %s %s , exec keyInputToChangeScene %s %s , exec branch action %s , exec transition action %s , wait %s seconds
Stage	gp	exec actor action %s	action	no	selectValue # %s separated by %s from %s , 3つのlist初期化、 exec pose action %s ; execActorAction
Stage	ee	hide all actors	—	no	execActorAction
Stage	eh	change skin of %s to %s	actorName , skinName	no	execActorAction
Stage	eR	show cover	—	no	selectValue # %s separated by %s from %s ; showMenu
Actor	it	isTimeBasedAction	—	no	—
Actor	actorWaitDef	wait for actor action %s seconds	seconds	no	—

transition・branch・input

target	ID	定義	引数	warp	直接の呼出し
Stage	ga	exec transition action %s	transitionName	no	exec transition reset, exec transition fadeUp, exec transition fadeOut
Stage	gf	exec transition fadeOut	—	no	—
Stage	gh	exec transition fadeUp	—	no	—
Stage	gj	exec transition reset	—	no	—
Stage	g]	exec branch action %s	branchName	no	selectValue...
Stage	hq	exec keyInputToChangeScene %s %s	keyIdList, sceneLabellist	no	Async Input
Stage	hu	touchInputToChangeScene %s %s	actorNameList, sceneLabellist	no	Async Input
Stage	hy	wait %s seconds	seconds	no	More Timers

6.3 主要な呼出し経路

起点	経路
green flag	初期化 → camera/pose停止 → actor 非表示 → showTitle
startStory	台本検証 → create sceneList → asset/actor 生成 → exec scene # %s with %s
scene実行	exec command %s %s → exec actionList → actionごとに exec action %s
Stage action	branch、transition、key/touch入力、wait などへdispatch
Actor action	runtime envelope 設定 → execActorAction → 対象 clone → 移動・見た目・音・時間 action
pose action	camera preview/pose 認識開始 → exec pose %s 反復 → 認識停止 → prompt 非表示
終了	stopStory → camera/pose停止 → actor 削除 → cover → menu

7. broadcast と状態遷移

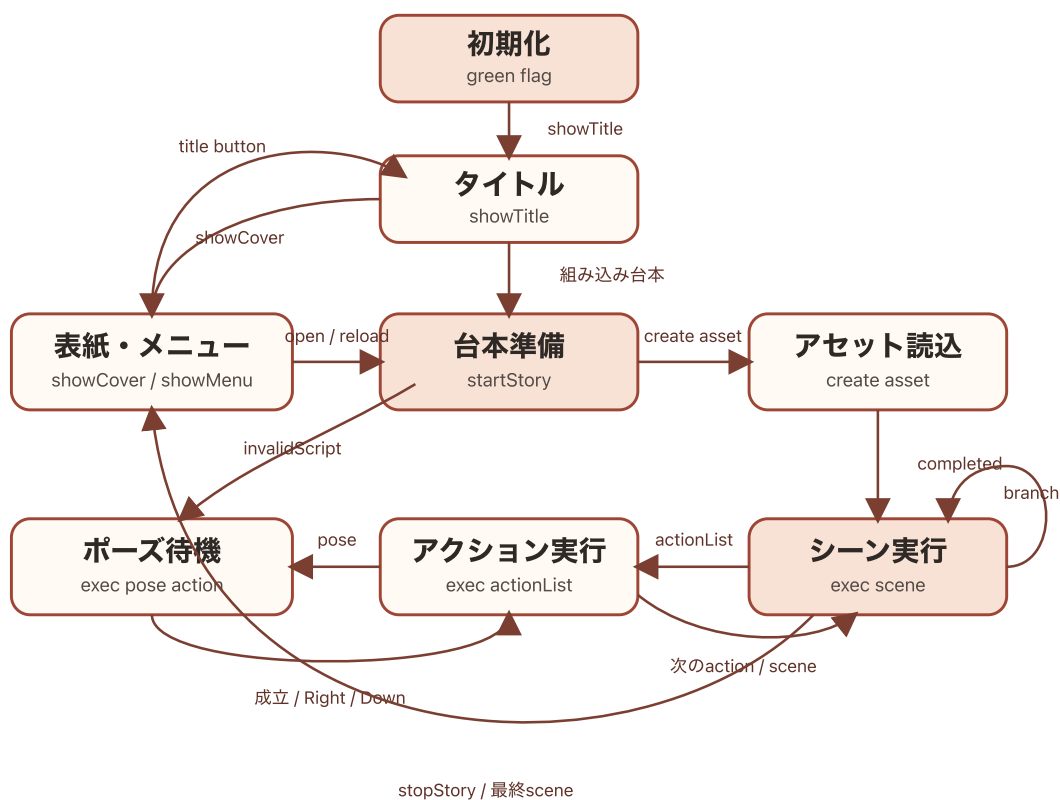
7.1 message 一覧

message	主な送信者	受信者	役割
<code>showPrompt</code>	<code>Stage</code>	<code>prompt</code>	操作・pose 案内を表示
<code>hidePrompt</code>	<code>Stage</code>	<code>prompt</code>	案内を非表示
<code>invalidScript</code>	<code>Stage</code>	<code>prompt</code>	台本エラーを表示
<code>hideMenu</code>	3つの menu button	3つの menu button	menu を一括非表示
<code>showMenu</code>	<code>Stage</code>	3つの menu button	利用可能な menu を表示
<code>startStory</code>	<code>Stage</code> , <code>openButton</code> , <code>reloadButton</code>	<code>Stage</code>	台本の解析・実行を開始
<code>stopStory</code>	<code>Stage</code>	<code>Stage</code>	実行を停止し cover へ戻す
<code>showCover</code>	<code>Stage</code>	<code>Stage</code>	cover を構築して menu を表示
<code>showTitle</code>	<code>Stage</code> , <code>showTitleButton</code>	<code>Stage</code>	title 状態へ戻す
<code>execActorAction</code>	<code>Stage</code>	<code>Actor</code>	action envelope を clone へ通知
<code>deleteAllActors</code>	<code>Stage</code>	<code>Actor</code>	全 clone を削除
<code>assetLoadingStarted</code>	<code>Stage</code> / Asset Manager	<code>Loading</code> , <code>LoadingBubbleAnchor</code>	Loading 表示を開始
<code>assetLoadingProgress</code>	<code>Stage</code> / Asset Manager	<code>Loading</code> , <code>LoadingBubbleAnchor</code>	進捗 costume と message を更新
<code>assetLoadingCompleted</code>	<code>Stage</code> / Asset Manager	<code>Loading</code> , <code>LoadingBubbleAnchor</code>	Loading 表示を終了
<code>stopKeyInput</code>	Async Input	<code>Stage</code>	key listener を停止
<code>stopTouchInput</code>	Async Input	<code>Stage</code>	touch listener を停止
<code>finishTimedActorAction</code>	<code>Stage</code> の Right/Down key hat	<code>Actor</code>	時間 action を確定状態へ進める

message	主な送信者	受信者	役割
debugTestCamera	TurboWarp editor からの手動送信	Stage	camera preview の診断

stopKeyInput と stopTouchInput は標準 broadcast block ではなく、Async Input へ渡した callback message です。 debugTestCamera は通常フローに送信元を持たない診断用 message です。

7.2 状態遷移



紙芝居アプリの主要状態遷移

主要状態はStageが所有します。UI表示そのものを状態の正本にせず、runtime variable、 broadcast、実行中の custom block から導出します。

状態	入口	主な出口
初期化	green flag	showTitle
title	showTitle	組み込み台本なら Stage click で startStory、それ以外は cover
cover/menu	showCover または stopStory	open/reload で startStory、title button で showTitle
台本準備	startStory	正常なら asset loading と scene 実行、異常なら invalidScript
asset loading	create asset	assetLoadingCompleted 後に scene 実行
scene 実行	exec scene # %s with %s	次 scene/branch、または最終 scene で stopStory
action 実行	exec actionList	Right で action 境界、Down で scene 境界、完了で次 action
pose 待機	exec pose action %s	pose 成立、Right/Down、エラーで action 実行へ戻る

skipMode は要求、skipContext は消費可能な境界です。要求は title、action、pose、scene の 該当境界だけが消費し、scene 開始、cover、stop で clear します。nextSceneLabel は key/touch listener が設定し、scene loop が label を index へ解決したあと削除します。

8. 検証

8.1 変更対象ごとのテスト

変更対象	主なテスト
builder API/CLI	test/builder.test.mjs
展開SB3の構造	test/sb3-project.test.mjs、test/skip-mode.test.mjs
内部仕様書の構造一覧	test/internal-specification.test.mjs
TurboWarp 実行結果	test/turbowarp-vm.test.mjs
入力、分岐、wait	test/async-input.test.mjs、test/register-branch.test.mjs、test/wait-action.test.mjs
文書、画像、ライセンス	test/docs-config.test.mjs、test/docs-images.test.mjs、test/documentation-license.test.mjs
公開物と汎用性	test/sb3-publication.test.mjs、test/build-freshness.test.mjs

8.2 標準チェック

```
pnpm lint
pnpm format
pnpm typecheck
pnpm test
pnpm run build
```

GitHub Actions は clean な Linux 環境で `pnpm install --frozen-lockfile`、`pnpm test`、`pnpm build` を実行します。ローカルで成功しても、未追跡ファイルや既存生成物へ依存していないことを CI で確認します。

`pnpm run build` は少なくとも次を生成・検証します。

- `dist/downloads/kamishibai.sb3`
- 一般文書の HTML/PDF
- 参加者向け・スタッフ向け体験会資料
- 公開サイトのリンク、画像、目次、PDF bookmark、favicon

SB3 または runtime を変更した場合は、生成 SB3 を TurboWarp で開いて次を手動確認します。

- 読込エラーがない
- green flag で title と menu が表示される
- 外部台本と組み込み台本の対象フローが開始できる
- pause、Space、Right、Down の進行が意図どおり動く
- Loading、画像、音声、テキストが正しく表示・再生される

内部構造を変更した PR では、`app/project.source.json` と本書の target、変数、message、hat、custom block 一覧を同時に更新します。

9. 公開

9.1 GitHub Pages

```
pnpm run deploy
```

`predeploy` がフル build を行い、成功した `dist/` だけを `gh-pages` へ公開します。公開後は top page、文書一覧、HTML/PDF、SB3 download を実際の URL から確認します。

問題がある場合は、直前の検証済み commit を checkout した clean な環境から再度 build・deploy します。生成済み `dist/` だけを手作業で修正しません。

9.2 npm パッケージ

公開済み version は変更・再利用できません。release ごとに新しい version と Git tag を使います。

1. `package.json`、`lockfile`、`src/builder/constants.js`、`README` の導入例を同じ version へ更新する。
2. clean な commit で標準チェック、フル build、公開内容の dry-run を実行する。

```
pnpm release:check
```

3. tarball のファイル一覧、license、size、CLI/API を確認する。
4. Git worktree ではなく通常の clean clone から公開する。
5. WebAuthn などの認証を完了して public package として公開する。

```
npm publish --access public
```

6. registry 反映後に metadata を確認する。

```
npm view @kubohiroya/tmpos-kamishibai@<VERSION> \
  version license dist-tags.latest dist.integrity --json
```

7. 一時ディレクトリへ公開版を導入し、CLI の `--version` と `@kubohiroya/tmpos-kamishibai/builder` の `import` を確認する。
8. 公開に使った確定 commit へ annotated tag を作り、GitHub Release を作成する。

公開後に問題が見つかった場合は対象 version を `npm deprecate` し、修正版を新しい patch version として公開します。公開済み tarball や tag を差し替えません。

10. トラブルシューティング

症状	確認と対応
<code>sb3:import</code> が置換を拒否する	<code>git status</code> と <code>git diff -- app</code> を確認し、 <u>toolchainの手順</u> に従う
<code>.app.rollback-*</code> や <code>.<出力名</code> <code>>.rollback-*</code> が残る	削除前に元出力と比較し、 <u>toolchainの失敗時の扱い</u> に従う
埋め込み拡張が追跡refと異なる	<code>pnpm sb3:extensions:status</code> で確認し、固定commitへ戻すなら <code>sync</code> 、更新するなら <code>update</code> を使う
PDF生成browserが見つからない	Chrome/Chromiumを導入し、必要なら <code>VIVLIOSTYLE_CHROME_PATH</code> を設定する
ローカルだけtestが通る	生成物と未追跡ファイルを確認し、 <code>clean clone</code> と <code>pnpm install --frozen-lockfile</code> で再現する
builderが既存出力を更新しない	エラーの <code>stage</code> 、asset名、URIを確認する。rollback領域が残っていないか確認する
公開直後に npm registry が404になる	同じ version を再publishせず、npm 公開page と registry の反映を待って確認する

復旧でGit履歴を破壊しません。公開済み変更は `git revert` または新しい修正PRで戻し、tagを移動しません。

11. ライセンスと秘密情報

対象	ライセンス
<code>docs/general/**</code>	CC BY-SA 4.0
<code>docs/workshops/**</code>	Copyright © 2026 Hiroya Kubo. All rights reserved.
上記以外で個別表示のない、本プロジェクトが著作権を持つソフトウェアと素材	MPL-2.0

詳細は [LICENSES.md](#)、[docs/general/LICENSE.md](#)、[docs/workshops/LICENSE.md](#) を参照してください。

第三者の画像、音声、font、model、機能拡張には個別のlicenseまたは利用条件が適用されます。builderで組み込む素材はasset manifestの `license` へ由来を記録します。許諾が確認できない 素材を本体またはsampleへ追加しません。

token、npm認証情報、秘密鍵、個人情報をrepository、SB3、台本、manifest、生成HTMLへ 記録しません。認証情報は環境変数、OSのkeychain、GitHub Secretsなど、公開物へ含まれない 仕組みで渡します。

12. 関連ドキュメント

- [03-user-guide.md](#) : アプリの利用方法と成果物の使い分け
- [04-dsl-manual.md](#) : 台本の構造と書き方
- [05-command-reference.md](#) : コマンドとactionの外部仕様
- [06-developer-guide.md](#) : setupと変更手順
- [history.md](#) : DSLとアプリの変更履歴
- [README.md](#) : プロジェクト全体の入口